

Task 2 Report: Dynamic Quantization of DistilBERT on SST-2

Abhishek Yadav — TalTech PhD Technical Evaluation

1. Task and understanding

The task is to measure how PyTorch dynamic quantization changes the inference efficiency of an SST-2 fine-tuned DistilBERT, and what that efficiency costs in accuracy. I first establish an FP32 baseline on four axes (parameters and MAC operations, latency, memory, and accuracy), then apply dynamic quantization in two different ways, repeat every measurement identically, and explain the differences. The focus of the discussion is on why each quantity changes, or fails to change, rather than on the differences alone.

2. Model, dataset and downstream task

The model is **distilbert-base-uncased-finetuned-sst-2-english** from Hugging Face. DistilBERT is a distilled version of BERT-base with six encoder layers, hidden size 768, twelve attention heads, and a feed-forward inner size of 3072. Each encoder block contains multi-head self-attention (four linear projections: query, key, value, output) followed by a two-layer feed-forward network with GELU; both sub-layers use residual connections and layer normalisation. Compared to BERT it drops the token-type embeddings, the pooler, and the next-sentence-prediction objective. A classification head (768→768 then 768→2) is applied to the first token. The checkpoint has 66,955,010 parameters: 23.83M in the two embedding tables, 42.47M in the six encoder blocks, and 0.59M in the head, spread over 38 nn.Linear and 2 nn.Embedding modules.

The dataset is SST-2 (Stanford Sentiment Treebank) from GLUE: binary sentiment classification of movie-review sentences with 67,349 train, 872 validation and 1,821 test sentences. GLUE test labels are withheld (they appear as -1), so accuracy cannot be computed on that split and all evaluation uses the 872-sentence validation split, which is standard practice for this benchmark. The metric is accuracy; macro-F1 is reported as a secondary check and tracks accuracy closely because the split is nearly balanced. Sentences are short (mean 25.2 tokens, max 55), so with a maximum length of 128 nothing is truncated.

3. Methodology

Quantization. All variants use **torch.ao.quantization.quantize_dynamic**. The weights of the selected module types are converted once, ahead of time, to INT8 and stored in a packed form suited to the CPU backend. Activations remain FP32 in memory. Immediately before each quantized linear layer, the input tensor is quantized to 8 bits from its current observed minimum and maximum. This on-the-fly step is the “dynamic” part, and it is why no calibration dataset is needed. The matrix product then executes as INT8 by INT8 into INT32 and the result is converted back to FP32. Everything that is not an nn.Linear module keeps running in FP32: the two attention matrix products (query by key, and attention by value), softmax, GELU and layer normalisation. The two ways chosen for this task were:

Method Q1 — per-tensor INT8 on linear layers, using **default_dynamic_qconfig**. All 38 nn.Linear modules receive one scale and one zero-point per weight matrix. This is the standard recipe from the official PyTorch BERT tutorial and serves as the reference method.

Method Q2 — per-channel INT8 on linear layers plus 8-bit embeddings, using **per_channel_dynamic_qconfig** for the linear layers and **float_qparams_weight_only_qconfig** for the embeddings. Compared to Q1, this adds one weight scale per output channel and additionally converts the two embedding tables to 8 bits with a per-row scale. The intuition is deliberate: per-channel scales should reduce rounding error at no cost in speed, and the embedding tables hold 36% of parameters but perform zero MACs, so quantizing them should reduce size without affecting latency. Both predictions are tested in Section 5.

Measurement protocol. Every model is measured on the CPU in a fresh Python process (so memory readings cannot contaminate each other) with one thread (slower in absolute terms but more stable than multi-threaded). Tokenisation is completed before any timer starts, so latency reflects only the model forward pass.

Parameters are counted including the weights packed inside quantized modules, which model.parameters() does not return, and are grouped by storage precision. MACs are counted with torch.utils.flop_counter.FlopCounterMode; because element-wise operations are excluded by convention, one MAC equals two FLOPs, so the counter’s output is divided by two. The count is independently checked against the hand-derived expression $n_{\text{layers}} \times (4Ld^2 + 2Ld_{\text{ff}} + 2L^2d) + d^2 + dC$, and counter and formula agree exactly, giving 5,587,404,288 MACs at $L = 128$. Latency is measured in two regimes: batch size one over all 872 sentences after 20 untimed warm-up passes (mean, standard deviation, median and 95th percentile are reported), and batch size 32 with dynamic padding, timed over three full passes of the dev set. Memory is reported three ways: theoretical weight bytes (parameters $\times$ bytes per element), the size of the saved state_dict on disk, and the peak resident set size of the Python process during inference, sampled every millisecond by a background thread. Accuracy is reported together with the agreement rate between each quantized model and FP32 and an exact McNemar test on the sentences where the two models disagree; on 872 sentences a single flipped prediction is 0.11 percentage points, so a statistical test is essential rather than decorative.

4. Experimental setup

Hardware. Intel Core i7-1165G7 (Tiger Lake, 4 physical / 8 logical cores, 2.80 GHz), 16.86 GB of RAM, running Windows 11. PyTorch reports the CPU capability as AVX-512, and the processor additionally supports AVX512-VNNI. All measurements are CPU-only because PyTorch dynamic quantization has no GPU kernels; the FP32 baseline is therefore measured on the CPU as well, so the comparison is fair.

Software. Python 3.12.2, torch 2.14.0+cpu, transformers 5.17.0, datasets 5.0.1, numpy 2.5.3, psutil 7.2.2. Backend: oneDNN. Attention: SDPA. Threads: 1. Batch sizes: 1 and 32. Max length: 128. Warm-up iterations: 20.

A limitation worth recording. This PyTorch build exposes only the oneDNN quantized engine; FBGEMM is not included. A third planned variant, FP16 dynamic quantization, could therefore not be evaluated, because `quantized::linear_prepack_fp16` is implemented only in FBGEMM. This is a property of the build (PyTorch is currently moving torch.ao.quantization to a separate torchao package, and the eager-mode API emits deprecation warnings in this version), not of the hardware, which supports AVX-512. INT8 quantization is unaffected and runs through oneDNN, which uses the same vector instructions.

5. Results

The two tables below summarise every measurement on the full 872-sentence SST-2 validation split. Q1 denotes per-tensor INT8 on linear layers, and Q2 denotes per-channel INT8 on linear layers with 8-bit embeddings. Table 1 reports the model cost; Table 2 reports memory, latency and accuracy. Figure 1 shows the latency comparison; Figures 2 and 3 show memory and the accuracy-latency trade-off.

Model	Params (M)	Stored as FP32 / INT8 (M)	GMACs per sentence	GMACs at L=128
FP32 baseline	66.96	66.96 / 0.00	1.076	5.587
Q1 (per-tensor INT8)	66.96	23.90 / 43.06	1.076	5.587
Q2 (per-channel INT8 + 8-bit emb.)	66.96	0.06 / 66.89	1.076	5.587

Table 1. Model cost. Quantization changes how parameters are stored, not how many exist, and it does not change the MAC count.

Model	Disk size MB (factor)	Peak RAM (MB)	bs=1 median ms (speedup)	bs=32 per-sentence ms (speedup)	Accuracy % (change)	McNemar p
FP32	267.9 (1.00x)	774	42.24 (1.00x)	51.79 (1.00x)	91.06 (ref.)	-
Q1	138.7 (1.93x)	828	13.63 (3.10x)	17.53 (2.95x)	90.02 (-1.03)	0.188
Q2	68.2 (3.93x)	739	15.00 (2.82x)	18.59 (2.79x)	90.14 (-0.92)	0.229

Table 2. Memory, latency and accuracy on the SST-2 validation split (CPU, 1 thread, oneDNN). McNemar p greater than 0.05 means the accuracy difference from FP32 is not statistically significant.

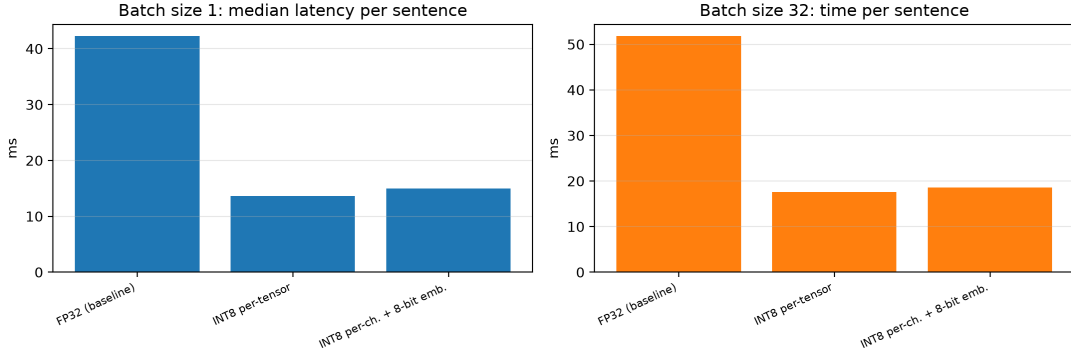


Figure 1. Latency per sentence at batch size one and batch size 32.

6. Discussion

Overall. Both quantized variants retain about 99% of the FP32 accuracy while being roughly two to four times smaller and three times faster. Throughput rises from 19.3 to 57.0 sentences per second with Q1. The quantization step itself takes about one second and requires no calibration data or retraining.

Why the model shrinks as it does. In Q1 only the 43.06 million linear weights move from four bytes to one byte, which predicts a drop from 267.8 to 138.6 MB (a factor of 1.93); the measured 138.7 MB matches. Q2 additionally converts the 23.83 million embedding weights, leaving only about 62 thousand FP32 parameters (layer-norm terms and biases), which predicts 67.1 MB against the 68.2 MB measured; the remaining 1.5% is scale-factor metadata plus pickle overhead. The size results are therefore fully explained by the parameter accounting in Table 1.

Why the model speeds up, and by more than the tutorial’s two-times figure. The MAC count is identical across all three models: dynamic quantization changes the precision of the arithmetic, not the amount of it. The gain has two sources. First, INT8 weights reduce memory traffic by a factor of four per matrix product. Second, AVX512-VNNI performs four INT8 multiply-accumulates per element slot in one instruction. Because 99.4% of the MACs on our short sentences occur inside the quantized linear layers, nearly all arithmetic is accelerated, which is why the observed 3.10-times speedup exceeds the roughly two-times figure quoted by the PyTorch BERT tutorial (measured on FBGEMM and older hardware). This advantage is length-dependent: at L = 128 the quantized share falls to 97.3%, because the attention products scale as L squared and stay FP32. On long inputs the speedup would shrink and the FP32 attention would eventually dominate.

The accuracy change is not statistically significant. The observed drops are 1.03 percentage points for Q1 and 0.92 points for Q2, but the exact McNemar test gives $p = 0.188$ and $p = 0.229$, so neither difference is distinguishable from noise at the 5% level. The two models are also behaviourally close to FP32, agreeing on 95.8% (Q1) and 96.1% (Q2) of sentences. Q1 corrects 14 sentences that FP32 got wrong and breaks 23, a net loss of nine out of 872. Q2 finishes one sentence ahead of Q1, which is consistent with the direction predicted by finer per-channel scales but is far too small to attribute to them on a dev set of this size.

6.1 Unexpected observations

Quantizing the embeddings makes the model much smaller but slightly slower. Q2 is 2.0 times smaller than Q1 on disk yet 10% slower at batch size one (15.00 ms against 13.63 ms). Embedding tables are lookup operations: they hold 36% of the parameters and contribute 0% of the MACs, so storing them in eight bits buys no arithmetic saving while adding a small dequantisation step per token. This is the cleanest result in the experiment: it shows directly that model size and latency are decoupled, and that where a parameter sits matters more than how many there are. For a memory-constrained deployment Q2 is the right choice; for a latency-constrained one, Q1 is.

Batched inference is slower per sentence than single-sentence inference (51.79 against 42.24 ms for FP32), the opposite of the usual expectation. The cause is padding. Batching pads every sentence up to the longest one in its batch, and although the dev-set mean is 25.2 tokens, batch maxima approach 55 tokens, so per-sentence computation roughly doubles. With a single thread there is no parallel capacity to absorb the extra work, so batching only adds padding cost without any compensating benefit. Length-sorted bucketing, or more threads, would remove most of this penalty. The INT8 speedup is preserved in both regimes (2.95 times batched and 3.10 times single), so the conclusion about quantization is unaffected.

The resident memory of the INT8 model exceeds FP32 even though its disk size is half (828 MB against 774 MB). The reason is that `quantize_dynamic(inplace=False)` builds the INT8 copy while the FP32 model is still resident, and deleting the FP32 model releases its memory to the C allocator but not necessarily to the operating system, so RSS records a high-water mark rather than current need. RSS is therefore a poor proxy for model size here, whereas the disk size and the theoretical byte count agree to within 1.5%. Consistently, Q2, which never holds two large float copies at once after conversion, shows the lowest peak of all three (739 MB). The practical lesson is that the memory benefit of quantization is realised when a quantized checkpoint is loaded, not when a float model is converted at run time.

6.2 Limitations

Q1 and Q2 differ in two respects at once (scale granularity and whether embeddings are quantized), so their difference cannot be attributed to either alone; an additional per-channel-only run (which the code supports via `--methods dq_int8_per_channel`) would isolate it. Latency was measured on a laptop CPU at a single thread count, so the absolute values are hardware-specific; only the ratios travel. The FP16 variant remains untested for the backend reason in Section 4.

6.3 Conclusion

Dynamic INT8 quantization of DistilBERT is close to free on this task: roughly three times faster, two to four times smaller, with an accuracy change that a McNemar test cannot distinguish from noise, obtained in about one second and without any training data. The results also show where its limits lie: it accelerates only what runs through linear layers, so its benefit shrinks as sequence length grows and the FP32 attention takes over.

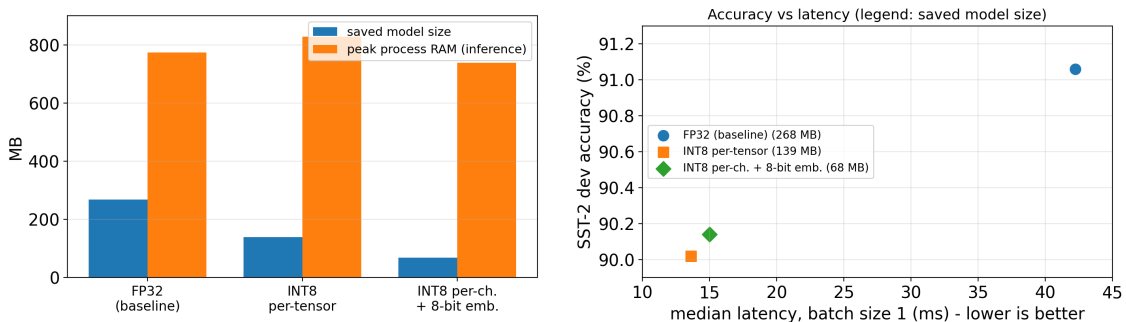


Figure 2 (left): disk size against peak process RAM; the two disagree because RSS records a high-water mark (Section 6.1). Figure 3 (right): accuracy against latency; both quantized models sit far to the left of FP32 at about one point below it.

7. Reproducibility

The full pipeline runs from one command, `python run_all.py`, which records the environment, counts MACs, benchmarks each model in its own process, and writes the tables and figures above. A second command, `python check_setup.py`, reports which quantization methods the local backend supports before any measurement is attempted. Every raw measurement, including per-sentence predictions and the complete environment record, is saved as JSON under `results/`. The README documents the setup, the command-line options, and the values that follow analytically from the architecture (66,955,010 parameters, 5.587 GMACs at $L = 128$, 267.8 MB in FP32), which were used to verify that the measurement code itself is correct.